

Portable Spec Kit — Spec-Persistent Development

Technical Overview — Architecture Reference Document

A comprehensive technical document covering the framework's architecture, user profile system, flow documentation, agent compatibility, security model, testing infrastructure, and deployment strategy.

v0.6.38 • May 2026

Author: Dr. Aqib Mumtaz, Ph.D.

Specialization: Computer Science — Artificial Intelligence

Version: v0.6.38

Tests: 1982 passing (1837 framework + 145 benchmarking), 95 sections

Executive Summary

Portable Spec Kit is a lightweight, zero-install, personalized spec-persistent development framework for AI-assisted engineering. It consists of a single markdown file (`portable-spec-kit.md`) that any AI coding agent reads to follow consistent engineering standards, manage specifications, track tasks, and maintain context across sessions.

Key numbers: 1 file, 5 AI agents supported, 10 core strengths, 8 source code templates, 16 flow documents, 1818 automated tests (1691 framework + 145 benchmarking), 92 test sections.

Key Highlights

Metric	Value
Current version	v0.6.29
Framework size	Single markdown file (912 lines, 54% reduced from 1998)
Skill files	17 (loaded JIT at peak attention)
Reliability scripts	12 (sync-check, critic-spawn, release, install-hooks, uninstall, code-review, scope-check, Jira, tracker)
Enforcement layers	3 (bash critic, sub-agent critic, git/Claude Code hooks)
Dependencies	Zero
Install time	Seconds (one curl command)
AI agents supported	Claude, Copilot, Cursor, Windsurf, Cline
Core strengths	13 (Lightweight, Portable, Personalized, Spec-Persistent, Agent-Agnostic, Context-Persistent, Secure, Non-Blocking, Self-Scaffolding, Self-Validating, Automated Test Generation, CI/CD Ready, Reliably Enforced)

Source code templates	8 (Web, Python, Mobile, Android, iOS, Full Stack, Full Stack + Mobile, Document)
Automated tests	1060 (915 framework + 145 benchmarking)
Research	SPD concept paper (5,300 words), benchmarking report, agentic communication discovery
Flow documents	16 step-by-step work flows
Project scenarios	9 (new, existing, monorepo, partial, cloned, subdir, returning, multiple, kit in one dir)
Profile setup	One-time, GitHub auto-detect, 3 preference questions, per-user workspace files
Versioning	Two-level: framework v0.x.x (patches) + release v0.x (milestones)

Version Changelog

v0.6 — AVACR Adversarial Framing + Sandbox Worktree + Peer-Exchange (April 2026)

Kit: v0.6.29

- **v0.6.0 — F70 Reflex evolves to AVACR (Adversarial Verbal Actor-Critic Refinement Loop):** Architectural shift within F70 Reflex from cooperative VACR to adversarial goals with collaborative convergence. *QA-Agent* goal = FAIL the release (hunt exhaustively across 14 dimensions; default DENIED). *Dev-Agent* goal = FOOLPROOF the release against QA's hunt (fix findings + harden sibling-class weaknesses). Convergence = QA hunted hard and cannot find a blocker, not "Dev says it's done."
 - **Physical sandbox isolation:** QA operates in a git worktree at `reflex/sandbox/qa-pass-NNN/` with `agent/AGENT.md` + `agent/AGENT_CONTEXT.md` physically removed. Dev's narrative files cannot bias QA's fresh-context investigation — enforced by filesystem, not prompt discipline. Dev retains full-repo read + write. Dual-rooted access (`REFLEX_QA_WORKTREE` for reads, `REFLEX_QA_PROJECT` for invocations).
 - **Peer-to-peer YAML exchange:** QA writes `findings.yaml` (per-finding: priority + dimension + blast_radius + confidence + reproducibility + escalation + citable_quote + regression_vector + standard + recommendation). Dev returns `dev-result.yaml` (fixed / rebuttals / routed_to_human / escalated / deferred). Human reads only `signoff.md` (GRANTED or DENIED with exit criteria) — never agent working output.
 - **14-dimension adversarial hunt:** functional / edge-case / cross-pipeline / workflow / research-backed-quality / superficiality / production-readiness / security (OWASP) / performance / architectural-compliance / documentation / test-quality / tech-debt / readiness-affirmation. Combined with **4-persona testing** (new-user / power-user / malicious-user / accessibility-user) per feature.
 - **Research capability:** QA may invoke WebSearch/WebFetch for RFCs, OWASP Top 10, published benchmarks, CVEs, academic papers. Paired with a **citable-quote honesty gate** — every finding requires `citable_quote.from` (file:line) + verbatim text that bash grep can verify. No hallucinated standards — `standard.url` must be real.
 - **4-bucket Dev diagnosis + Bucket-D rebuttal:** A (doc wrong) / B (code wrong) / C (both inconsistent) / D (QA misread — requires counter-citation in `dev-result.yaml` rebuttals). Dev does not dismiss findings.
 - **Human-arbitration escalation:** findings tagged `escalation: human-arbitration` routed to `@<human>` ; Dev does not auto-fix cross-cutting / ADL-amendment concerns.
 - **Flaky-vs-deterministic distinction:** non-reproducing findings trigger root-cause investigation (race / ordering / isolation), not patch-retry.
 - **Coverage declaration:** QA writes `coverage.md` enumerating tested × dimensions AND untested surface with reasons. Signoff covers tested only.

- **Regression-vector re-execution:** every finding preserves exact invocation + expected assertion. Re-executed verbatim each pass. Failure upgrades priority one level (catches surface-patch vs root-cause fixes).
 - **CHANGELOG truth-check mandate:** every bullet in current version's CHANGELOG must have a test case; unverifiable claims → QA-REL-{N} blocks signoff.
 - **Sibling-class hardening in Dev:** on any finding, Dev audits for same-class issues across the codebase and fixes siblings as separate atomic commits. Fixing the class, not just the instance, is what separates fool-proof from whack-a-mole.
 - **Consistency sweep:** cycle → pass across ~30 references in live scripts + CSV column header. Refine → Reflex (3 pre-existing stragglers including real install/update URL bugs). Remediation → Refinement .
 - **Paper title:** *"Adversarial Verbal Actor-Critic Refinement (AVACR): Multi-Agent Verbal Reinforcement Learning with Asymmetric Goals for Spec-Grounded Post-Release Program Repair"*. Target venues: ICSE / FSE / ASE. "Adversarial" refers to inference-time asymmetric goals / red-teaming, NOT adversarial training (no gradient updates).
 - **Tests:** 1077 framework + 145 benchmarking = 1222 (unchanged count; 2 Section 60m assertions updated for AVACR formal name + black-box discipline).
 - **Bug fixes surfaced during sweep:** reflex/install-into-project.sh was installing into non-existent \$TARGET/refine ; reflex/update.sh KIT_RAW_BASE pointed at 404 URL. Both fixed.
- **v0.6.1 — Kit self-evolution loop + MINIMAL template tier + 14 framework-gap closures:** Large dual-track release closing the empirical-kit-evolution loop end-to-end and adding a third response-template tier for latency cuts.
- **N33 adopter-side auto-submit:** new reflex/lib/auto-submit.sh dispatcher at end of every pass. Three modes — manual (default), prompt (y/n with 10s timeout), auto (fire-and-forget). Four guards — consent marker, pass filter (verdict in {NEEDS_FIX, REVOKED} OR kit/meta scope), 24h rate limit, anonymise. Opt-in via bash reflex/run.sh --enable-auto-submit --i-understand-privacy-implications . Config in reflex/config.yml under upstream_submission: . Audit log at reflex/history/auto-submit-log.csv .
 - **N38 maintainer-side intake automation:** new reflex/lib/intake.sh plus .github/workflows/kit-intake.yml . Weekly Monday 09:00 UTC GitHub Action fetches open avacr-eval issues, parses kit/meta-scope findings, drafts agent/tasks/Gxx-<slug>.md task files from template, opens an intake-review PR with intake-review-pending label. Parser is deterministic Python (no LLM on intake path). Manual run via bash reflex/intake.sh .
 - **N44/N45 general-purpose runner + kit-loop:** mode-less bash reflex/run.sh [--target <path>] where kit-identity detection drives routing automatically. New --kit-loop chains prep-release plus self-test with max 3 iterations via state machine at agent/.release-state/kit-loop-state.yml . Convenience wrappers reflex/kit-loop.sh and reflex/self-test.sh (retired in v0.6.2 per entry-consolidation; use

`bash reflex/run.sh` and `bash reflex/run.sh --self-test`). `--help / -h` prints usage.

- **Trace continuity across passes (QA reads prior trace):** QA-Agent prompt now mandates reading the prior pass's `findings.yaml` (structured evidence with `regression_vector.invocation_verbatim` + `expected_assertion`) and `signoff.md` (verdict + deferred decisions) at Phase 1, plus the cross-pass `REFLEX_EVAL_TRACE.md` register. `spawn-qa.sh` surfaces all three paths in the task-file preamble.
- **Per-pass canonical trace (Q1):** each pass commits `findings.yaml` (structured, one entry per finding with `citable_quote` + `regression_vector` + `recommendation`) plus `signoff.md` (verdict GRANTED / DENIED, blocking findings, deferred decisions). Cross-pass `reflex/history/REFLEX_EVAL_TRACE.md` register auto-aggregates every finding from every pass via `reflex/lib/update-eval-trace.sh` into one reviewable index (id · severity · scope · status · one-line summary). The earlier 8-section `AVACR_EVAL_TRACE.md` narrative was consolidated into this three-file model — reduces duplication across artifacts.
- **v0.6.2 — Dev-branch isolation:** every reflex pass runs Dev on a dedicated `reflex/dev-cycle-NN-pass-NNN` branch (or `reflex/dev-standalone-pass-NNN` for ad-hoc runs). User's main untouched during the pass; fast-forward merge on GRANTED + auto-delete; unhappy branches retained (last 3) for diagnosis.
- **v0.6.2 — Protected-files write-ban:** 3-layer enforcement that `agent/AGENT.md` and `agent/AGENT_CONTEXT.md` are never modified by reflex findings (prompt constraint + `gates.sh` diff check + `psk-sync-check.sh` pre-commit hook on reflex dev branches).
- **v0.6.2 — Autoloop generalization:** bare `bash reflex/run.sh` invocation is now the autoloop entry point (aliases `--autoloop` / `--loop` / `--kit-loop`) chaining prep-release + reflex pass + iterate until GRANTED, identical across kit / new project / existing project. (Earlier `reflex/autoloop.sh` wrapper retired per same-release entry-consolidation.)
- **v0.6.2 — History retention:** `reflex/lib/prune-history.sh` + `history_retention` config bound disk growth (defaults: `pass_dirs_keep` 10, `dev_branches_keep` 3, `qa_sandbox_keep` 3). Register + `summary.csv` kept forever; archived passes marked `_(archived)_` in register.
- **v0.6.2 — Concurrency + empty-pass:** `agent/.release-state/reflex.lock` prevents parallel reflex corruption; 0-finding passes shortcut to GRANTED with no Dev spawn.
- **v0.6.2 — Intake parser update:** `reflex/lib/intake.sh` reads `findings.yaml` YAML block from GitHub issue bodies (matches v0.6.2+ submission format).
- **v0.6.3 — Autonomous reflex run validation:** mechanical patch minted from an autonomous end-to-end reflex run on the v0.6.2 consolidation. QA surfaced 11 findings (3 CRITICAL including dead-code branch regex in the write-ban); Dev landed 11/11 fixes in 6 atomic commits on the dev branch, fast-forward merged to main.

- **v0.6.4 — Convergence-based stopping (N56):** replaces the naïve "max 3 iterations" cap with real convergence signals: GRANTED, REGRESSION, findings_floor, plateau (patience_passes), fix-rate drop (min_fix_rate), safety_cap as escape hatch. Configurable in `reflex/config.yml` → `convergence:` .
- **v0.6.4 — --loop --resume collision fix:** deferred flag resolution in `run.sh` ; `--loop --resume` now routes to `loop-resume` mode instead of hijacking to `resume-qa` .
- **v0.6.4 — gates.sh cache-order bug:** rft-cache cleared at gate-runner entry so sequential gates don't see stale data from the test gate.
- **v0.6.4 — Token tracking (reflex/lib/track-tokens.sh):** per-pass tokens aggregated into `reflex/history/token-usage.csv` with `tokens_per_finding` metric + per-pass and per-cycle budget warnings.
- **v0.6.4 — Iteration auto-chain on gate-fail:** spurious gate failures no longer abort the autoloop; only REGRESSION + verdict/findings-trend signals halt.
- **v0.6.5 — Kit-bootstrap integrity gate (4-layer defense):** prevents un-installed-kit projects from running prep-release or reflex. New `agent/scripts/psk-bootstrap-check.sh` (7 structural checks C1-C7: framework file + entry-point symlink, `.portable-spec-kit/config.md` , core 6 kit scripts, cached skills, pre-commit hook, all 9 `agent/*.md` pipeline files, test harness) wired into `psk-release.sh` Step 0, `reflex/lib/preconditions.sh` Gate 0a, and QA-Agent Dimension 16. Catches projects scaffolded manually by non-kit-aware agents (Copilot, plain Claude) that skipped `install.sh` . Bypass: `PSK_BOOTSTRAP_CHECK_DISABLED=1` . Kit-self auto-detected — skips symlink/config checks for the kit repo.
- **v0.6.5 — Register format improvements:** `reflex/lib/update-eval-trace.sh` parses cycle from pass-name (`cycle-NN-pass-NNN`) when `.cycle-meta` has stale `cycle=0` , adds `coverage.md` drill-down, extracts "Tests exercised:" summary line per pass — surfaces PASS-side data, not just failures.
- **v0.6.6 — Reflex Phase 0 pre-compute (claim-driven probing + reference-state diff + assumption surfacing):** closes the meta-gap surfaced in v0.6.5 review where reflex QA ran 3 GRANTED passes on an infrastructurally-empty project because the probe set was a closed 16-dim checklist. Moves reflex from "fixed checklist" to "probe-set derived from the project itself," while keeping the 16-dim checklist as a safety net. **Mode A** (`reflex/lib/extract-claims.sh`): walks docs and extracts every claim with `probe_type` + `probe_target` . **Mode B** (`reflex/lib/state-diff.sh` + `reflex/reference-state/speckit-project.yaml`): reference definition of complete speckit project; deltas pre-classified by severity; CRITICAL auto-promote to findings. **Mode C** (in `qa-agent.md`): every pass writes `assumptions.md` ; unverifiable assumption = MAJOR finding. Both helpers run in `spawn-qa.sh` BEFORE sandbox creation. Dev-Agent expanded: fix findings + build unfulfilled claims + remediate state-diff deltas. Deterministic bash (<1s, <100 tokens pre-compute) saves per-pass LLM budget for semantic verification.
- **v0.6.7 — Reflex 7-layer Senior/Principal-level QA system:** the user must NOT be QA of their own project. Reflex now operates with a 7-layer Senior/Principal-level QA

system that derives all probes from the project's spec pipeline + kit toolkit + running system — zero human-curated lists. **Layer 1** (`reflex/lib/check-rft-integrity.sh`): R→F→T pipeline integrity per [x] feature (Rn map + criteria block + design plan + non-trivial test + TASKS mark). **Layer 2** (`reflex/lib/doc-code-diff.sh`): bidirectional doc↔code consistency. **Layers 3-5/7**: qa-agent.md mandates (behavioral 1+5+3+1 per feature / test-quality audit / cross-feature integration / external CVE+OWASP+deprecation research). **Layer 6**: spawn-qa.sh captures `psk-sync-check` , `psk-doc-sync` , `test-release-check` , `psk-code-review` outputs as research input. Senior/Principal-level philosophies codified in qa-agent.md + dev-agent.md prompts. 16-dim checklist preserved as safety net; CRITICAL signals from Layer 1/6 short-circuit it. Architecture plan committed at `agent/design/f70-reflex-senior-engineer-qa.md` . Companion Dev plan at `agent/design/f70-reflex-senior-engineer-dev.md` . Layer 4 hardening log shipped (`reflex/history/hardening-log.md` + `reflex/lib/log-hardening.sh`).

- **v0.6.8 — Reflex 7-layer full implementation: Layer 3/5/7 QA seed-helpers + Layer 7 Dev ADL extraction:** completes the v0.6.7 architecture by shipping deterministic helpers for Layers 3, 5, 7 (QA) and Layer 7 (Dev). All 7 layers now have deterministic-bash seed-helpers wherever possible — LLM creative work goes to genuine reasoning. **scaffold-behavioral-tests.sh** generates 1+5+3+1 test-plan skeletons per [x] feature with criteria pulled from `### F{N}` blocks. **identify-integration-probes.sh** finds feature-pair candidates from design-plan + criteria-block cross-references. **external-research.sh** scans manifests (package.json / requirements.txt / go.mod / Dockerfile) and seeds CVE / framework-changelog / OWASP queries; includes stack-specific recent-vuln hints (e.g. prompt injection per OWASP LLM Top 10). **auto-extract-adl.sh** drafts ADL entries from Dev's commit-body rationale post-pass. `spawn-qa.sh` Phase 0 produces 8 pre-populated artifacts. Field-tested on kit-self: 70 test-plan skeletons in <1s, 77 candidate pairs found.
- **v0.6.9 — The kit is audited and improved by the reflex it built:** the architecture built across v0.6.5-v0.6.8 was designed to find drift in user projects. v0.6.9 turned that exact machinery against the kit itself. The same QA-Agent that catches bugs in your code caught real bugs in the code that builds the QA-Agent — Phase 0 exposed 180 mechanical drift items (170 R→F→T pipeline breaks + 10 doc↔code drifts) the kit's own existing mechanical gates had passed clean for months; QA filed 5 findings; Dev closed 2 inline (live `kit-loop-state.yml` → `loop-state.yml` rename incompleteness from v0.6.2 + Layer-4 hardening shipping new `PSK018` per-feature pairwise check that would have caught 69-of-70-criteria-blocks-deleted under the existing shallow gate). Layer 5 hub-suppression added (77→13 pairs after F64+F70 hubs filtered). Field-test artifacts committed in `reflex/history/field-test-v0.6.8/` as permanent evidence. *A tool sufficient to audit user projects must be sufficient to audit the project that built it — v0.6.9 proved that proposition.*
- **v0.6.29 — User-project reflex correctness — 5 fixes G19-G23 from searchsocialtruth field-test:** first user-project field-test results land via the kit-evolution protocol. Reflex run on searchsocialtruth (v0.1.1) surfaced 5 kit-machinery

bugs filed as `scope: kit`. **ADR-041 (G23)**: kit-self detection uses kit-only directory discriminator (examples/starter + examples/my-app + tests/sections + install.sh + agent/PHILOSOPHY.md) — replaces broken symlink-vs-real-file heuristic. Cascade fix unblocking correct routing for every other QA-KIT-* finding. **ADR-042 (G19)**: sandbox cp-fallback enumerates dynamically — catches `src/`, `app/`, `components/`, `packages/` on Next.js / monorepo projects. **ADR-043 (G20)**: rft-integrity regex matches h2-h6 R-rows. **ADR-044 (G21)**: release-check `check_test_relevance()` advisory check. **ADR-045 (G22)**: doc-code-diff `IS_KIT_SELF` skip clause for kit-installed paths on user projects. 1756 tests passing (1611 framework + 145 benchmarking); 9 N84 regression sub-tests cover all 5 fixes. [source: `avacr-eval-searchsocialtruth/cycle-01/pass-001`]

- **v0.6.27 — Symmetric self-evolution + Tier 3 auto-probe-synthesis + bookend release ceremony + finding-closure helper + cycle-id rule relaxation**: major kit-evolution release closing the v0.6.7+ residual auto-evolving-QA backlog plus a structural blind spot the user surfaced after Mode C iteration (kit was biased toward addition and blind to consolidation). **ADR-038 — Tier 3 auto-probe-synthesis**: `agent/scripts/psk-blind-spot-synthesize.sh` (~280 LOC) reads `qa-blind-spots.md` open entries, classifies target into 5 buckets, scaffolds PR-style proposals at `agent/tasks/proposed/Gxx-<slug>.md`. **ADR-039 — Reflex bookend release ceremony**: iter 1 = prep-release · iters 2+ skip ceremony entirely · GRANTED runs ONE final refresh-release. N-iteration cycle now runs ceremony exactly 2× (was N×). **ADR-040 — Symmetric Self-Evolution (P9)**: 5 layers — `agent/PHILOSOPHY.md` P9 principle · `psk-coverage-overlap-check.sh` (~210 LOC) · `/optimize` cat 14 (probe redundancy) · Phase 6 evolution-gauntlet Gate G · QA Dim 24 (3 probes filing `QA-OVERLAP-NN`) · Phase 5 mandate ≥1 audit-coverage OVERLAP observation · `OL-NNN` registry parallel to `BS-NNN` · `psk-close-finding.sh` closes findings without YAML hand-edit. Cycle-id rule relaxed to verdict-agnostic (closes DENIED-then-externally-fixed trap). Kit bug fix: `psk-bootstrap-check.sh` C7 kit-self detection. Backlog cleanup 48→17. 1879 tests (1734 framework + 145 benchmarking); 22/20 sync-check; new N80 (12) + N81 (6) + N82 (15) regressions plus 8 cycle-04 regressions. *Closes auto-evolving-QA loop; kit now self-detects probe redundancy alongside probe gaps.*
- **v0.6.26 — Per-cycle pass numbering + summary.csv schema v5 + reflex self-test convergence**: closes user-reported "cycle 2 showing pass 3,4,5 instead of 1,2,3" bug. Pass numbers reset per cycle (each cycle starts fresh at `pass-001`); composite key `(cycle, pass)` gives unique row identity in `summary.csv` (schema v5 adds `cycle` column). Lifts ADR-037 (reflex self-test 8-finding closure landing in v0.6.25 patches) into a formal release.
- **v0.6.25 — Post-merge soak gauntlet + Tier 1/4/5 auto-evolving QA**: ADR-035 ships `agent/scripts/psk-soak-schedule.sh` (Phase 6 (e) post-merge 48h soak, deferred from v0.6.24). ADR-036 closes Tier 1 (BS-NNN registry seeding), Tier 4 (auto-promote OPEN → CLOSED on Gxx merge), Tier 5 (count-based gauntlet probes for blind-spot density tracking). ADR-037 (reflex self-test 8-finding closure landing in patch series).

- **v0.6.24 — Self-evolving-plan gap-closure (hygiene release):** three rounds of audit-and-close on the v0.6.16-23 plan that surfaced 9 sub-deliverables headline commits had claimed-shipped but were missing. Round 1 closed 7 phase artifacts. Round 2 closed `install.sh` gaps. Round 3 closed 5 flow-doc omissions. Phase 6 (e) deferred to v0.6.25. *Pure hygiene; lesson: companion artifacts must commit alongside headline deliverables.*
- **v0.6.13 — Nested per-cycle history layout + Dim 23 + cycle-summary aggregator + autoloop convergence:** closes the meta-gap class user surfaced in cycle-01 trace audit (5 issues + 4 sub-gaps). Formalizes **Dim 23 — Auditor-output hygiene** with five new probes (register hygiene · cost data persistence · parallel-directory disambiguation · cross-surface terminology consistency · self-output validation loop) so QA self-evolves to catch this class going forward. Migrates reflex history from flat `cycle-NN-pass-NNN/` to nested `cycle-NN/pass-NNN/` ; sandbox mirrors. Per-cycle parent dir hosts a `summary.md` aggregator (`reflex/lib/cycle-summary.sh`). Drops the hard 3-iter autoloop cap in favor of convergence-based stopping (GRANTED · REGRESSION · findings_floor · plateau · fix-rate drop); `max_iterations_safety` default 20 is escape hatch only. First successful Dev-Agent run end-to-end: cycle-01-pass-004 closed `QA-NOISE-RE-01` bucket A, all 6 mechanical gates green, fast-forward auto-merge. 3-layer QA→Dev contract enforcement closes "Dev never spawned on partial QA timeout" pattern. *Auditor-output hygiene formalized; reflex history reshaped per-cycle; autoloop runs to convergence.*
- **v0.6.12 — Reflex iter-1 convergence anchor + close all 15 v0.6.11 audit findings:** anchor version for the reflex iter-1 convergence cycle. Consolidates 8 refresh-release commits from v0.6.11 baseline. All 15 audit findings (5 pre-existing ARBs + 10 iter-1 QA findings) resolved or properly deferred. Major perf win: persistent test-ref cache cold 54s → warm 5s = 10.9× speedup. Dim 17 cost/perf audit + Dim 18 philosophy self-audit added to QA prompt + Dev mirror principles 8/9. `pass_score` (Dim 17 ranking) v3 summary.csv schema. Option C thematic test-suite split: `tests/lib.sh` + `tests/sections/{01-infrastructure,02-pipeline,03-reliability,04-reflex}.sh` + thin orchestrator. *Iter-1 self-audit complete — kit closed every finding it surfaced about itself.*
- **v0.6.11 — Reset allowlist + iter-aware refresh-release optimization:** two kit-infrastructure improvements driven by usage friction during v0.6.10 wrap-up. (1) `reflex/lib/reset.sh` switched from hardcoded glob lists to allowlist-based `find` traversal with `HISTORY_KEEP=("hardening-log.md")` . Symmetric `--reset-hardening` flag for true clean-slate. (2) `reflex/lib/loop.sh` Phase 1 now branches on `ITER : iter 1 = prepare release` (version bump), `iter 2+ = refresh release` (no bump — captures Dev's auto-merged fixes through same critic gates at ~50% cost). +7 new test assertions. Doc-coverage backfill closes 24 cumulative `psk-doc-sync.sh` gaps from v0.6.5-v0.6.10.
- **v0.6.10 — Closing v0.6.9 ARBs: kit's own RFT debt backfilled + PSK018 strict by default:** acted on the v0.6.9 field-test verdict. Backfilled all 55 missing `### F{N}` acceptance criteria blocks in `agent/SPECS.md` (F1–F55, 3-5 testable bullets each, ~200

lines), closing `QA-PIPE-FT-01-ARB` . Flipped `PSK018` (per-feature pairwise check in `psk-sync-check.sh`) advisory → strict-by-default; future commits adding an `[x]` feature without matching block fail pre-commit. Opt-out: `PSK_FEATURE_CRITERIA_STRICT=0` . Layer 1 RFT integrity on kit-self: 55 C2-criteria breaks → 0.

- **14 kit framework gaps closed (G1-G15 less G11):** parser + regex consistency across `reflex/lib/` (G1+G2+G5+G6); `test-templates.md` skill with live-uvicorn fixture (G7); G11 pre-shipped via canonical trace template; rate-limit escape-hatch docs (G13); serial-execution trade-off disclosure in `f70-reflex` design (G14); `spawn-qa` SQLite artifact purge (G9); per-pass token accounting with v2 summary.csv schema (G15); `file-bugs.sh` schema validator + dim-floor enforcement (G8); machine-readable coverage YAML block (G10); minimum-per-dimension probe count rule (G12); preconditions accept reflex-install commits (G3); `--recover` diagnostic plus `reflex/lib/recover.sh` for partial Dev state (G4).
- **MINIMAL template (3rd tier) for short answers:** breadcrumb + border + headline + optional one-paragraph body + arrow. Target ~2s gen time, ~80-100 tokens. Use for yes/no, status checks, short factual recalls, simple acknowledgments. BRIEF remains default for multi-dimensional answers; DETAILED on explicit depth request.
- **Auto-dispatch rule:** trigger table maps answer shape to template tier — agent picks, user does not specify. Override phrases — `"shorter"` drops a tier, `"more depth"` promotes one, `"raw reply"` strips scaffolding.
- **Writing Style sub-section (5 editorial rules):** one idea per sentence · periods over em-dashes · drop semicolons · cut parenthetical asides unless load-bearing · one voice per reply.
- **Breadcrumb rules 6a + 6c + 6d:** trailing `✓` -hint of most-recently-closed sibling (6a); skip breadcrumb on consecutive MINIMAL replies in same node (6c); arrow footer optional in MINIMAL with no next action (6d). Rule 6b (IDs-only compact) shipped then reverted per user feedback — clarity beat the ~15-token savings.
- **Pre-send self-check:** mandatory 4-anchor check (breadcrumb, border, template body, arrow footer) run silently before every reply. No exemptions for "let's discuss" or "status update" or post-compact replies.
- **3-tier long-session scalability:** T1 auto-collapse 3+ consecutive `✓` siblings into `Nx-Ny ✓ <summary>` ; T2 default-render active chain on `"where was I"` (full tree on `"show full stack"`); T3 mid-session chapter rotation at ≥15 KB or ≥80 nodes into `.session-archive/YYYY-MM-DD-chapter-NN.md` .
- **Promotion rule:** closed `✓` session-stack node with durable project work (code/docs changes) prompts for `agent/TASKS.md` entry before session end.
- **Pre-session read list updated:** `.session-stack.md` added to the session-start read order (item 4 of 11).
- **Skill trigger for "where was I" / "losing context" / "conversation-stack":** auto-loads `.portable-spec-kit/skills/session-trace.md` .

- **Skill trigger for "integration tests" / "FastAPI" / "live-server":** auto-loads the new `.portable-spec-kit/skills/test-templates.md`.
- **Rule-revision post-mortem meta-rule:** an ADL entry is required in `agent/PLANS.md` when any framework rule is revised ≥ 3 times in a single session (learning-from-churn).
- **Unpushed-commit advisory in psk-sync-check:** `--full` mode warns at ≥ 10 unpushed commits on current branch. Threshold configurable via `PSK_UNPUSHED_WARN`.
- **Signoff discipline:** `signoff.md` lists blocking findings explicitly; verdict is GRANTED only when zero CRITICAL + zero MAJOR blocking. Kit maintainer routes `scope: kit / meta` findings to `agent/tasks/Gxx-*.md` via `file-bugs.sh`.
- **Prompts rewritten:** QA-Agent + Dev-Agent prompts updated for adversarial framing, sandbox mechanism reference, citable-quote gate, and trace continuity across passes.
- **Paper title update:** *"Adversarial Verbal Actor-Critic Refinement (AVACR)"* — see v0.6.0 bullet above for full title and disambiguation.
- **SPD paper v2 §7.6 cross-reference:** paper methodology section now cross-references AVACR evaluation harness.
- **Design plan §4-5 and §18:** `agent/design/f70-reflex.md` §4 (QA-Agent), §5 (Dev-Agent), §18 (serial execution trade-off) updated for v0.6.0 shape.
- **Adversarial-goal reframing:** see v0.6.0 bullet above — this v0.6.0 delta narrows the framing to explicit convergence criterion.
- **Sandbox mechanism:** see v0.6.0 bullet above — v0.6.0 adds dual-rooted access reference documentation.

v0.5 — Full Pipeline + Reliability Architecture (April 2026)

Kit: v0.5.1 — v0.5.23

- **Full development pipeline (F63–F69):** REQS → SPECS → PLANS → DESIGN → TASKS → RELEASES with `RESEARCH.md` support. Jira integration (`psk-jira-sync.sh`), auto code review (`psk-code-review.sh`), scope drift detection (`psk-scope-check.sh`), project config, onboarding tour, kit self-help, requirements + research pipeline, skill-based architecture.
- **v0.5.8 — Reliability Architecture (3-layer enforcement):**
 - **Layer 2A — Bash critic:** `psk-sync-check.sh` runs 18 deterministic checks (version consistency, test count, flow doc count, feature count, $R \rightarrow F \rightarrow T$ gate, script perms, required dirs, CHANGELOG/RELEASES version, ARD content, AGENT.md stack, README Latest Release, README install list, README agent table, README flow table, flow doc content, critic prompts meta-check, secrets scanning). Mode auto-detection (kit/node/python/go/rust/generic/custom). `--quick` (<500ms, version+tests only) and `--full` (all 18 checks) modes. Structured error codes PSK001–PSK017. Silent on clean. Sensitive data exclusion (`.env` , `*.pem` , `*.key` , `credentials.*`). Emergency bypass via `PSK_SYNC_CHECK_DISABLED=1`.

- **Layer 2B — Sub-agent critic:** `psk-critic-spawn.sh` file-based protocol (`critic-task.md` + `critic-result.md`). Three templates: flow-docs verification, release-notes verification, final-validation comprehensive sweep. 5-iteration cap with escalation. CURRENT/STALE structured output.
- **Layer 3 — Hooks:** `.claude/settings.json` with PostToolUse hook (warns on edit drift, silent on clean). `.git/hooks/pre-commit` blocks commits that fail sync-check. Hooks run outside agent control — agent cannot bypass. `psk-install-hooks.sh` installer detects Husky/pre-commit framework, wraps existing hooks without overwriting.
- **Release pipeline rewritten (`psk-release.sh`):** 10 steps with artifact gates. Validation moved to Step 9 (final gate) from Step 4. Step 6 (Version Bump) fully automated — reads current version from `AGENT_CONTEXT.md` , computes next patch, seds all files, verifies via sync-check. Steps 4 and 8 agent-required with artifact verification (mtime + content checks). State persisted in `agent/.release-state/` .
- **End-user distribution (`install.sh`):** One-command install: `curl -fsSL .../install.sh | bash` . Downloads 12 scripts + 17 skill files + templates. Auditable (prints summary before execution). `--yes` for non-interactive. `--from` for local testing. Symlinks for all agents (`CLAUDE.md`, `.cursorrules`, `.windsurfrules`, `.clinerules`, `.github/copilot-instructions.md`). Clean uninstall via `psk-uninstall.sh` (preserves user pipeline files).
- **Context reduction:** Core framework file 1998 → 912 lines (54% reduction). 894 lines extracted to 17 skill files loaded JIT at peak attention (mitigates "Lost in the Middle" attention gradient). New skill files: `hooks-and-critics.md` , `self-help.md` , `ci-setup.md` , `config-details.md` , `init-process.md` , `project-setup.md` , `onboarding-tour.md` , `dashboard.md` , `multi-agent.md` , `jira-integration.md` (plus existing templates, python-environment, source-structures, profile-setup, document-generation, test-release-check-template, release-process).
- **Research document:** `agent/research/plans/r1-agent-reliability-architecture.md` (3600 lines, 24 sections). Synthesizes 5 academic papers (Lost in the Middle, ReAct, Process Reward Models, Reflexion, Self-Consistency), production framework analysis (LangGraph, Temporal, OpenClaw), CI/CD patterns (Make, Bazel, Terraform), and Google Research multi-agent critic work (PaperVizAgent, ScholarPeer). Documents token optimization strategy (architecture is net token-negative vs baseline when optimized).
- **v0.5.21–v0.5.23 — F70 Reflex (Adversarial Verbal Actor-Critic Refinement Loop, AVACR):** Post-prep-release automated adversarial QA + auto-fix loop with asymmetric goals (QA fails / Dev foolproofs). Fresh-context sandboxed *QA-Agent* (Critic, senior-engineer auditor, 14-dimension hunt, 4-persona testing, research-grounded findings with citable-quote honesty gate, `AGENT.md` + `AGENT_CONTEXT.md` excluded from sandbox worktree) reads the full speckit pipeline (`REQS/SPECS/PLANS/DESIGN/RELEASES` + docs + running system) and adversarially exercises every SPECS acceptance criterion + CHANGELOG claim through the project's real public surface. Files peer-exchange `findings.yaml` with priority + blast-radius + confidence calibration. Fresh-context *Dev-*

Agent (Actor, peer fixer with adversarial foolproof goal, 4-bucket diagnosis + sibling-class hardening + Bucket-D grounded rebuttals + human-arbitration routing) fixes atomically with per-commit mechanical gates, 1 finding = 1 commit, max 3 retries before [~] escalation. Pass-over-pass regression-vector re-execution catches surface-patch fixes. `reflex/history/summary.csv` tracks surprise_density trend. Precondition: HEAD must be a prep-release commit. Counters the "lost in the middle" failure mode where trivial Dev-written tests pass against stub implementations. `reflex/install-into-project.sh` deploys machinery into any speckit project with auto-detected gates. Section 60m adds 45 tests. First pass on the kit surfaced 5 real gaps existing structural gates couldn't see; Dev-Agent auto-fixed 7 of 8 findings with atomic commits. Docs: `docs/work-flows/17-reflex.md` ; design plan `agent/design/f70-reflex.md` .

- **v0.5.23 — Full-Surface Documentation Coverage Analyzer (`psk-doc-sync.sh`):** CHANGELOG → full doc-surface coverage tool that extracts every feature from the current-minor CHANGELOG entry and verifies it's reflected across all 5 documentation surfaces: `agent/*.md` , `docs/work-flows/*.md` , `docs/research/*.md` , `ard/*.html` , `README.md` . Classifies each feature COVERED (≥2 surfaces) / PARTIAL (1 surface) / MISSING (0 surfaces) with surface-legend `[AFPDR]` and heuristic-suggested target doc per gap. Mandatory at release Step 4; `STEP_4_FLOW_DOCS` and `STEP_9_VALIDATION` critic prompts wired to treat every MISSING line as STALE. `--strict` flag returns exit 1 for CI use. Section 60k adds 19 tests. First run surfaced 27+ MISSING features from v0.5.x as backlog for v0.5.23+ — the gate is now in place to prevent regression while that backlog is closed.
- **v0.5.10 — Dual-Gate Reliability Across All Executable Workflows:** Generic `psk-validate.sh <workflow>` helper terminates every workflow (release, feature-complete, init, reinit, new-setup, existing-setup) with the same bash + sub-agent pair. 5 new critic templates: `FEATURE_COMPLETE` (R→F→T + ADL), `INIT` (all 9 agent/* populated from code), `REINIT` (no content loss during re-sync), `NEW_SETUP` (scaffold coherence), `EXISTING_SETUP` (non-destructive adoption). `psk-release.sh` Step 9 now delegates to `psk-validate.sh release` . Framework MANDATORY rule: agent runs the helper at end of every executable workflow. Section 60 (25 tests) verifies the wiring end-to-end.
- **v0.5.12 — Confidence Hardening (7 items):** Step 4 of release upgraded from mtime-only to per-flow-doc critic verdict (`STEP_4_FLOW_DOCS` template required; every flow doc must have explicit `CURRENT:` by filename, missing coverage flagged, any `STALE:` blocks). Bypass audit log at `agent/.bypass-log` records every `PSK_SYNC_CHECK_DISABLED=1` or `PSK_CRITIC_DISABLED=1` use (timestamp, user, workflow); `sync-check --full` surfaces warning for any in last 24h. 5 workflow orchestrators (`psk-feature-complete.sh` , `psk-init.sh` , `psk-reinit.sh` , `psk-new-setup.sh` , `psk-existing-setup.sh`) add workflow-specific preflight: feature-complete enforces R→F→T + design plan + no TODO stubs; reinit snapshots agent/* byte counts to detect content loss; existing-setup snapshots non-kit file sizes to detect destructive edits.
- **v0.5.13 — Secret Scanning (PSK011):** `check_secrets` added as 12th deterministic check in `psk-sync-check.sh --full` . Scans tracked files via single `git grep` for 12 high-signal credential patterns: AWS access/secret keys, GitHub PAT/fine-grained/OAuth/

App tokens, Anthropic, Google, Slack, Stripe live/restricted, private key PEM headers. Placeholder-aware (`paste-your` , `example.com` , `placeholder` , etc. excluded) + path exclusions (`.env.example` , `node_modules/` , `tests/fixtures/` , binaries). Masked output prevents leakage to logs. PreCommit hook now blocks any commit containing tracked secrets — closes the gap between framework security rule and mechanical enforcement. Section 60d adds 11 tests.

- **v0.5.15 — Verbatim-Quote Critic (sub-agent honesty gate):** Closes the architectural ceiling on sub-agent honesty. Every `CURRENT: verdict` in `critic-result.md` now requires a `QUOTE:` line with a verbatim string (≥ 20 chars) from the named file — `bash grep -F` verifies the quote actually exists. Fabricated or missing quotes exit 3 and block the workflow. All 6 critic templates updated. Sub-agent honesty: $\sim 70\% \rightarrow \sim 85\%$. Overall confidence: $\sim 96\%$.
- **1222 tests** (1077 framework across 74 sections + 145 benchmarking). Section 59: Reliability Architecture (20 tests). Section 60: Dual-Gate Validation + Behavioral (28 tests). Section 60c: Workflow Orchestrators + Step 4 + Bypass Audit (21 tests). Section 60d: Secret Scanning (11 tests). Section 60e: Distribution Completeness (7 tests). Section 60f: Verbatim-Quote Critic (12 tests).

v0.4 — CI/CD Pipeline + Spec-Based Test Generation (April 2026)

Kit: v0.4.1 — v0.4.11

- **GitHub Actions CI:** `ci.yml` runs all 3 test suites on every push and PR. `release.yml` verifies tag matches framework version on v* tag push.
- **Framework CI/CD rules:** `ci.yml` template with stack-aware commands (Jest/pytest/Go/Bash), New Project Setup Step 7.5, Existing Project Setup checklist CI item, CI & Community Contributions section (4 rules).
- **Spec-based test generation (8 rules):** SPECS origin detection (forward vs retroactive), per-feature `### F{n}` acceptance criteria format, stub generation trigger, stack-aware stub formats, stub completion gate.
- **check_stub_complete():** added to `tests/test-release-check.sh` (kit copy + template) — blocks release if test stubs have unfilled TODO markers.
- **Community files:** PR template, bug report + feature request issue templates. CI badge in README.
- **Cross-platform sed fix:** `test-spd-benchmarking.sh` now runs on Ubuntu (GitHub Actions).
- **746 tests** (601 framework across 48 sections + 145 benchmarking). Section 42: CI/CD (29 tests). Section 43: Spec-Based Test Generation (15 tests).
- **v0.4.5:** 2 new flow docs (11-existing-project-setup, 12-cicd-setup). All 12 flow docs validated at 63-char box alignment. Section 19 expanded to cover all 12 flows. ARD flow diagrams section added. Prepare release rule updated: flow docs first, mandatory ARD audit, mandatory PDFs.

- **v0.4.6:** Flow docs reordered logically (onboarding 01-06, session 07-10, development 11-12, release 13). New `05-project-init.md` — explicit `init / reinit` commands. Renamed: `12-project-lifecycle.md`, `13-release-workflow.md`. `init / reinit` added as explicit command signals with full flows. Test Execution Flow extracted as named section — referenced by all test-running commands. Version bump BEFORE push rule added. 746 tests (601 framework + 145 benchmarking).
- **v0.4.7 — Team Intelligence Layer (F58–F62):** Progress Dashboard (trigger words → inline burndown with progress bars, BY CONTRIBUTOR for @username projects). Multi-Agent Task Tracking (@username ownership syntax, per-user task view, delegation/unassign protocol, cross-agent coordination via TASKS.md). Persistent Memory Architecture (named concept: 6 agent files = Persistent Memory, 5 properties: Durable/Shared/Portable/Team-scale/Auditable). Architecture Decision Log (ADL promoted to `## Architecture Decision Log` top-level in PLANS.md, ADR-NNN format, immutable history, supersede pattern). AI-Powered Onboarding (commit `agent/` for team/OS projects, clone → briefed, agent-agnostic). New flow doc: `14-team-collaboration.md`. Sections 44–48. 752 tests (607 framework across 48 sections + 145 benchmarking).
- **v0.4.8 — Release command fix:** `prepare release` corrected — steps 1–7 only, no commit, no push. Release summary shows GitHub/Tag as ⌚ pending. `prepare release` and `push` is the explicit command that commits and pushes. `13-release-workflow.md` flow doc updated (7-step gate, corrected trigger table, 5 new edge cases). README commands table updated.
- **v0.4.9 — Release flow hardened + Kit Self-Validation + Portability Gate:** ARD audit rule changed to `ard/*.html` glob (no hardcoded filenames). WeasyPrint commands use loop form. Section 41 + 2 new tests: kit `agent/AGENT_CONTEXT.md` Version+Kit match; `CHANGELOG.md` built-over range includes current version. Section 41: 28→33 tests. ARD Guide version fix. Section 49: Kit Self-Validation (8 tests) — example symlink validity, Progress Summary, template compliance, root/Projects copy sync. Section 50: Framework Portability (11 tests) — no machine-specific paths, no hardcoded tool binary paths, WeasyPrint loop form, ARD glob, sync.sh/flow docs/ARD HTML clean. +7 symlink tests in Sections 6/7/11. 885 tests (740 framework across 58 sections + 145 benchmarking).

v0.3 — Framework Hardening + R→F→T Traceability (April 2026)

Kit: v0.3.1 — v0.3.27

- **R→F→T traceability chain:** every done feature requires test reference in SPECS.md before release. `tests/test-release-check.sh` distributed as kit template, auto-created on project setup.
- **15 new enforcement sections:** SPECS.md staleness check, RELEASES.md trigger rule, scope change recording (DROP/ADD/MODIFY/REPLACE), no-slip task rule, pipeline sync, pre-release consistency gate (Section 41).
- **Release Process rule — full 8-step sequence:** tests → counts → version bump → PDFs → RELEASES.md → CHANGELOG.md → GitHub release → tag update. Explicit signals only.

- **Release command aliases:** `update release` = alias for `prepare release` (full 8-step); `refresh release` = re-test + sync current release without bumping version.
- **Release notes scope rule:** only include changes committed and visible in public repo; never mention excluded files (private docs, research papers).
- **Simplified versioning:** aligned patches (v0.N.x = release v0.N). All specific version examples replaced with generic `v0.N.x / v0.1.4→v0.1.5` so rules never go stale. GitHub release title = minor version (v0.N — Title) matching CHANGELOG.
- **README hardening:** Critical Scenarios table (8 situations), Development Guidelines section, "first methodology native to the AI era" framing.
- **746 tests** (601 framework across 48 sections + 145 benchmarking). All passing.

v0.2 — Versioning + Project Scenarios (April 2026)

Kit: v0.2.1 — v0.2.9

- **Aligned versioning:** v0.N.x patches belong to release v0.N (e.g. v0.2.x = release v0.2). Auto-restructure agent files when framework version changes on pull.
- **Existing project setup:** guide don't force — scan, show checklist, user picks changes. Never rename/move/delete without approval.
- **9 project scenarios:** new, existing, monorepo, partial, cloned, subdir, multiple projects, returning, kit in one subdir only.
- **Project directory confirmation:** list directories, user picks or types new path before creating agent/ files.
- **Git rule:** check `.git/` before committing — each project can be its own repo.
- **Version-based TASKS.md:** tasks grouped under release headings (v0.0 Done, v0.1 Done, v0.2 Done, v0.3 Current, Backlog).
- **RELEASES.md:** kit version range per release (v0.0.x for v0.0, v0.1.x for v0.1).
- **AGENT_CONTEXT.md:** tracks Kit version — compared on session start to trigger restructure.
- **442 framework tests across 41 sections + 145 benchmarking tests (587 total).** Added: Python/Conda env rules (27 tests), README validation (3 tests), requirements-to-delivery flow checks, 15 enforcement sections (staleness, RELEASES trigger, scope changes, no-slip, pipeline sync, security, push rules, git, existing projects, scenarios, retro fill, context persistence across 10 disruptions, R→F→T traceability chain).

v0.1 — Personalized Profile + Flow Documentation (April 2026)

Kit: v0.1.1 — v0.1.9

User Profile

- **Profile moved to** `.portable-spec-kit/user-profile/user-profile-{username}.md`

- **Global home directory storage** — asked once, works everywhere
- **Per-user workspace files** — committed, team-ready
- **Username detection via** `git config user.name` (slugified)
- **First-time setup:** GitHub fetch + 3 questions with RECOMMENDED defaults + Enter to skip
- **New project:** show profile, keep or customize, save to workspace
- **Returning session:** workspace exists → silent; only global → show/confirm
- **12 edge cases handled** (no gh CLI, empty files, multiple users, special chars)
- **Cross-OS:** macOS/Linux (`~`) + Windows (`%USERPROFILE%`)

Flow Documentation

- **16 flow documents created in** `docs/work-flows/`
- **Each flow:** title, trigger condition, step-by-step diagram, edge cases, files involved
- **Context Management rule:** update flows after implementation/tests

Framework

- **10 core strengths** (added Personalized, Secure, restored Self-Scaffolding)
- **8 source code structure templates** (added Mobile, Android Native, iOS Native, Full Stack + Mobile)
- **Strict security rule** — never expose API keys even if explicitly asked
- **Context triad:** context + flows + tests must always match code
- **Guide moved from** `docs/` **to** `ard/`
- **Deleted** `setup.sh` — agent handles profile creation

Agent-Agnostic

- **Removed all** `.claude/` **paths,** `noreply@anthropic.com` , `Claude Opus`
- **Generic** `Co-Authored-By: AI Agent <noreply@ai-agent.dev>`
- **27 agent-switching tests verifying zero data loss**

Testing

- **242 tests across 22 sections (was 122)**
- **New sections:** User Profile directory, first-time setup, new project, returning session, edge cases, flow documentation, ARD directory, context management

v0.0 — Initial Release (March 2026)

- **CLAUDE.md framework — portable spec-persistent development**
- **README with spec-kit comparison table**
- **CONTRIBUTING.md with PR guidelines**
- **PDF Quick Guide**
- **Two example projects (starter + my-app)**
- **GitHub repo created (aqibmumtaz/portable-spec-kit)**
- **Sync script for publishing**
- **122 automated tests**

Table of Contents

- 1. Architecture Overview**
- 2. User Profile System**
- 3. Work Flows**
- 4. Multi-Agent Support**
- 5. Security Model**
- 6. Source Code Templates**
- 7. Spec-Persistent Development Pipeline**
- 8. Testing Infrastructure**
- 9. File Structure & Purposes**
- 10. Deployment & Syncing**

1. Architecture Overview

The framework is a single markdown file that AI agents read as their instruction set. It creates a layered system:

```
Layer 1: portable-spec-kit.md           ← Framework (portable, shared)
Layer 2: .portable-spec-kit/           ← User profile (per-user, committed)
Layer 3: agent/AGENT.md                ← Project rules (per-project)
Layer 4: agent/AGENT_CONTEXT.md        ← Living state (per-session)
Layer 5: agent/SPECS → PLANNING → TASKS → TRACKER ← Dev pipeline
```

Design Principles

Principle	Implementation
Zero install	One curl command downloads the file, symlinks for all agents
Personalized	GitHub profile auto-detect, 3 preference questions, adapts behavior
Agent-agnostic	Symlinks: CLAUDE.md, .cursorrules, .windsurfrules, .clinerules, copilot-instructions.md
Non-blocking	Code first, specs retroactively — agent tracks silently
Context-persistent	AGENT_CONTEXT.md updated every session, picks up after weeks
Self-validating	Agent tests everything, fixes before presenting, 98%+ coverage target
Secure	Never expose API keys — even if asked. Absolute rule, no exceptions.

File Hierarchy

```
workspace/
├─ portable-spec-kit.md           ← Framework (source)
├─ .portable-spec-kit/           ← Kit config
│   └─ user-profile/
│       └─ user-profile-{username}.md ← Per-user profile (committed)
```

[illegible]

2. User Profile System

Purpose

Tells the AI agent **WHO** it's working with — affecting response depth, technical language, analogies, and autonomy level.

Storage

Location	Purpose	Committed?
<code>~/.portable-spec-kit/user-profile/user-profile-{username}.md</code>	Global — asked once	No (home dir)
<code>workspace/.portable-spec-kit/user-profile/user-profile-{username}.md</code>	Per-project, per-user	Yes

Lookup Order

- 1. Workspace** `.portable-spec-kit/user-profile/user-profile-{username}.md`
- 2. Global** `~/.portable-spec-kit/user-profile/user-profile-{username}.md`
- 3. Neither** → **first-time setup**

Username Detection

`git config user.name` → **slugified (lowercase, spaces → dashes)**. Used for filename. `gh api user` **used only for greeting name/bio**.

Profile Format

```
# User Profile
> Auto-created on first session. Edit anytime.

- **Name** — Education. Expertise.
- Communication style: {selected or custom}
- Working pattern: {selected or custom}
- AI delegation: {selected or custom}
```

Preference Options

Question	Options
Communication	(a) direct and concise ← RECOMMENDED (b) direct, data-driven, comprehensive (c) conversational and collaborative
Working pattern	(a) iterative — starts brief, expands ← RECOMMENDED (b) plan-first — full specs before code (c) prototype-fast — build quick, refine
AI delegation	(a) AI does 70%, user guides 30% ← RECOMMENDED (b) AI does 90%, user reviews 10% (c) 50/50 collaboration

Edge Cases

Scenario	Handling
No gh CLI	Ask name/expertise manually
GitHub name empty	Use login as fallback
GitHub bio empty	Ask for education/expertise
Empty profile file	Treat as missing, run setup
Multiple users (team)	Each user has own file
Special chars in name	Slugified for filename
Agent can't write	Show content, user creates manually
User skips all questions	Recommended defaults applied

3. Work Flows

14 documented flows in `docs/work-flows/` covering every system interaction:

#	Flow	Trigger	Key Actions
01	First Session in New Workspace	New workspace	Environment detection → WORKSPACE_CONTEXT → auto-scan
02	User Profile Setup	First time using kit	GitHub fetch → 3 questions → save global + workspace
03	New Project Setup	Creating new project	Show profile → keep/customize → scaffold agent/ + dirs
04	Existing Project Setup	First-time kit onboarding	Detect state → scan → fill agent/ → checklist → apply selected
05	Project Init & Reinit	<code>init / reinit</code> commands	Confirm dir → deep scan → create/update agent/ files → show summary
06	CI/CD Setup	New/existing project setup	Stack detect → ci.yml template → badge → branch protection
07	Returning Session	Coming back to project	Load profile → read context → summarize → continue
08	Agent Switching	Changing AI agent	All agents read same file via symlinks → zero data loss
09	Profile Customization	Different prefs per project	Show CURRENT + RECOMMENDED → save local override
10	File Management	Agent encounters managed file	Create / restructure / leave-as-is — never lose content
11	Spec-Persistent Development	Any workflow — specs persist	SPECS → PLANS → TASKS → RELEASES — living specs, never block
12	Project Lifecycle		

		Client requirements received	R→F traceability → scope changes → F→T coverage → handoff
13	Release Workflow	prepare release command	Tests → counts → version bump → PDFs → publish → tag
14	Team Collaboration	Multi-agent / team project	@username tagging → per-user view → delegation → progress dashboard

Context Management Rule

After completing implementations or running tests: update `agent/AGENT_CONTEXT.md` **to reflect current code status. Also update** `docs/workflows/` **if any work flow changed, and update test files if new behaviors were added. Context, flows, and tests must always match the actual state of the code.**

Flow Diagrams

Step-by-step box diagrams for all 16 flows. All box lines are exactly 63 display characters wide. Connectors:  **→**  **. No tree-style standalone arrows or spacing lines.**

01 — First Session in New Workspace

When: User opens an AI agent in a workspace for the first time — no `WORKSPACE_CONTEXT.md` exists.

```
1. CHECK PROFILE
workspace/.portable-spec-kit/user-profile/ → FOUND?
~/.portable-spec-kit/user-profile/ → FOUND?
Neither → run User Profile Setup flow

NOTE: Profile setup and workspace scan are independent
If user skips profile → apply defaults, proceed
Never block workspace scan waiting for profile
```

2. DETECT ENVIRONMENT

OS, Node version, Python version, tools installed
→ populate Environment & Tools section

3. SCAN WORKSPACE

List all directories → identify projects
→ populate Workspace Overview table
Announce: "Scanning workspace – found X projects..."

4. CREATE WORKSPACE_CONTEXT.md

Sections:

- Workspace Overview (table of all projects)
- Environment & Tools
- Key Conventions
- Last Updated

5. CREATE AGENT/ DIRS FOR PROJECTS WITHOUT THEM

For each project found without agent/ dir:

- Create agent/ with 6 template files
- Apply File Creation/Update Rule to README.md

Show kit status: 🔍 Understanding your project...

6. WORKSPACE READY

"Workspace scanned. X projects found."
"What would you like to work on?"

02 — User Profile Setup

When: First time using the kit on any machine — no profile exists anywhere.

1. DETECT USERNAME

git config user.name → slugified (lowercase, spaces→-)
Filename: user-profile-jane-smith.md

2. FETCH GITHUB PROFILE

gh api user → full name, bio (if gh authenticated)

- └ gh available → "Welcome, Jane Smith!"
- └ gh not authenticated → ask name + expertise manually
- └ gh not installed → ask manually

3. ASK 3 PREFERENCE QUESTIONS

Q1: Communication style?

- (a) direct and concise ← RECOMMENDED
 - (b) data-driven, comprehensive with tables
 - (c) conversational and collaborative
- Enter = use recommended

Q2: Working pattern?

- (a) iterative ← RECOMMENDED
 - (b) plan-first
 - (c) prototype-fast
- Enter = use recommended

Q3: AI delegation?

- (a) AI does 70%, user guides 30% ← RECOMMENDED
 - (b) AI does 90%, user reviews 10%
 - (c) 50/50 collaboration
- Enter = use recommended

4. CONFIRM PROFILE

Jane Smith — B.S. CS.
Communication: direct
Working pattern: iterative
AI delegation: AI does 70%

"Looks good? (Enter = yes, or type changes)"

5. SAVE PROFILE

~/portable-spec-kit/user-profile/user-profile-{u}.md
workspace/portable-spec-kit/user-profile/...
→ won't ask again on this machine

6. CONTINUE TO PROJECT SETUP

✓ Profile ready — proceeds to workspace/project scan

03 — New Project Setup

When: User asks to create a new project or enters a directory without agent/files.

1. LOAD USER PROFILE

workspace/.portable-spec-kit/ → global → setup flow
Show: "Welcome back, Jane! Setting up new project."
Keep or customize for this project?

2. CREATE PROJECT STRUCTURE (immediately – no questions)

agent/ + 6 files, src/, tests/, docs/, ard/
input/, output/, cache/
README.md, .gitignore, .env.example
tests/test-release-check.sh (R→F→T validator)

3. REPORT TO USER

Show structure created – wait for user to start

When user is ready:

4. SPECS DISCUSSION

Write agent/SPECS.md – requirements, features
Write per-feature acceptance criteria (### F{n})
Agent generates test stubs for forward-flow features

5. TECH STACK → user approves

Write agent/PLANS.md – architecture, phases

6. INITIALIZE STACK

Install deps, update .gitignore, assign port
If Python → conda environment selection flow

7. CREATE CI WORKFLOW

.github/workflows/ci.yml (stack-aware test command)
Always includes test-release-check.sh as final step
Add CI badge to README.md
"CI will run on every push and PR."

8. START DEVELOPMENT

Update agent/TASKS.md → begin building

05 — Project Init & Reinit

When: User says "init" (full project scan, any kit status) or "reinit" (re-scan to sync stale agent files). Agent never auto-triggers either command.

```
"init" — Full Scan (New / Partial / Mapped)
  Confirm project directory
  Deep scan — config files, src/, top-level dirs
  Create / update all 6 agent/ files from scan
  Present optional changes checklist
  Apply selected → show init summary ✅
```

```
"reinit" — Re-scan (already Mapped)
  Read agent/AGENT.md + AGENT_CONTEXT.md as baseline
  Deep scan — source files, config, directory structure
  Update AGENT.md — only fields that changed
  Rebuild AGENT_CONTEXT.md from current codebase
  SPECS.md staleness check (completed tasks vs features)
  PLANS.md vs code check — flag stale architecture
  Show reinit summary ✅
```

10 — File Management

When: Agent encounters any auto-managed file (WORKSPACE_CONTEXT.md, README.md, agent/ files) and needs to decide whether to create, update, or leave it.

```
ENCOUNTER A MANAGED FILE
(WORKSPACE_CONTEXT.md / README.md / agent/* files)
```

```
Does file exist?
├─ NO → CREATE from template
│     Fill in known details (project name, stack)
└─ YES → Does file match template?
        ├─ YES → LEAVE AS-IS — no changes
        └─ NO → RESTRUCTURE:
                Read ALL content
                Map to template sections
                Reorganize structure
```

RETAIN every detail

Kit Version Update — Full Scan Flow (runs when kit version changes on pull):

1. RESTRUCTURE AGENT/ FILES

Compare each file to current template
Reorganize structure, retain all existing content
Update **Kit:** field to new version

2. STALE FIELD NAME SWEEP

grep -r "Framework versions:" → rename to "Kit:"
grep -r "***Framework:**" → rename to "***Kit:**"
Check ALL files: agent/, docs/, examples/, templates

3. SCAN PROJECT CODEBASE

Read source files, config files, directory structure
(package.json, requirements.txt, Dockerfile, etc.)
Update AGENT.md: stack, tech, ports — from actual code
Update AGENT_CONTEXT.md: phase, done, next — from state

4. SHOW COMBINED SUMMARY

"Portable Spec Kit updated to vX.X."
"Your project: [stack] · [phase] · [X tasks pending]"
"Agent files updated: AGENT.md, AGENT_CONTEXT.md"
"What's new in vX.X: [CHANGELOG entries]"
Continue conversation — zero interruption

09 — Profile Customization

When: User wants different preferences for a specific project.

1. SHOW CURRENT PROFILE

Load global: ~/.portable-spec-kit/user-profile/...

Jane Smith — B.S. CS.
Communication: direct
Working pattern: iterative
AI delegation: AI does 70%

"Keep or customize? (Enter = keep)"
(a) Keep as-is (b) Customize for this project

User selects (b)

2. ASK 3 QUESTIONS – each shows CURRENT and RECOMMENDED
Q1: Communication style?
Q2: Working pattern?
Q3: AI delegation?
Press Enter to keep current value for each

3. CONFIRM CHANGES
Show summary with changed vs unchanged fields
"Looks good? (Enter = yes, or type changes)"

4. SAVE TO WORKSPACE – global unchanged
workspace/.portable-spec-kit/user-profile/...
"This workspace uses local profile from now on"

07 – Returning Session

When: User opens an AI agent in an existing project – coming back after hours, days, weeks, or months.

1. LOAD FRAMEWORK
Agent reads portable-spec-kit.md (via symlink)
Check kit version: Framework Version in kit
vs **Kit:** in agent/AGENT_CONTEXT.md
├ Same → proceed normally
└ Different → run Kit Update flow (see flow 04)

2. SHOW KIT STATUS (once at session start)
✅ Spec Kit: Project mapped (vX.X.X) – reading context
⚠ Spec Kit: Partial context – filling in gaps...
🔍 Spec Kit: Understanding your project – scanning...

3. LOAD PROFILE
1. workspace/.portable-spec-kit/user-profile/ → FOUND?
└ Yes → load silently, no questions


```
2. ~/.portable-spec-kit/user-profile/ → FOUND?
   └ Yes → show profile, keep or customize → save WS
3. Neither → run User Profile Setup flow first
```

```
4. READ PROJECT CONTEXT (in this order)
   1. agent/AGENT.md      – project rules, stack
   2. agent/AGENT_CONTEXT.md – living state
   3. agent/TASKS.md      – current tasks
   4. agent/PLANS.md      – architecture
```

```
5. GREET + SUMMARIZE
   "Welcome back, Jane! Here's where we left off:"
   - Version and phase from AGENT_CONTEXT.md
   - Tasks completed vs pending from TASKS.md
   - Last decision + next step
   "Want to continue with [next task]?"
```

```
6. WORK
   User gives instructions → agent works
   Track all tasks in TASKS.md (no-slip rule)
```

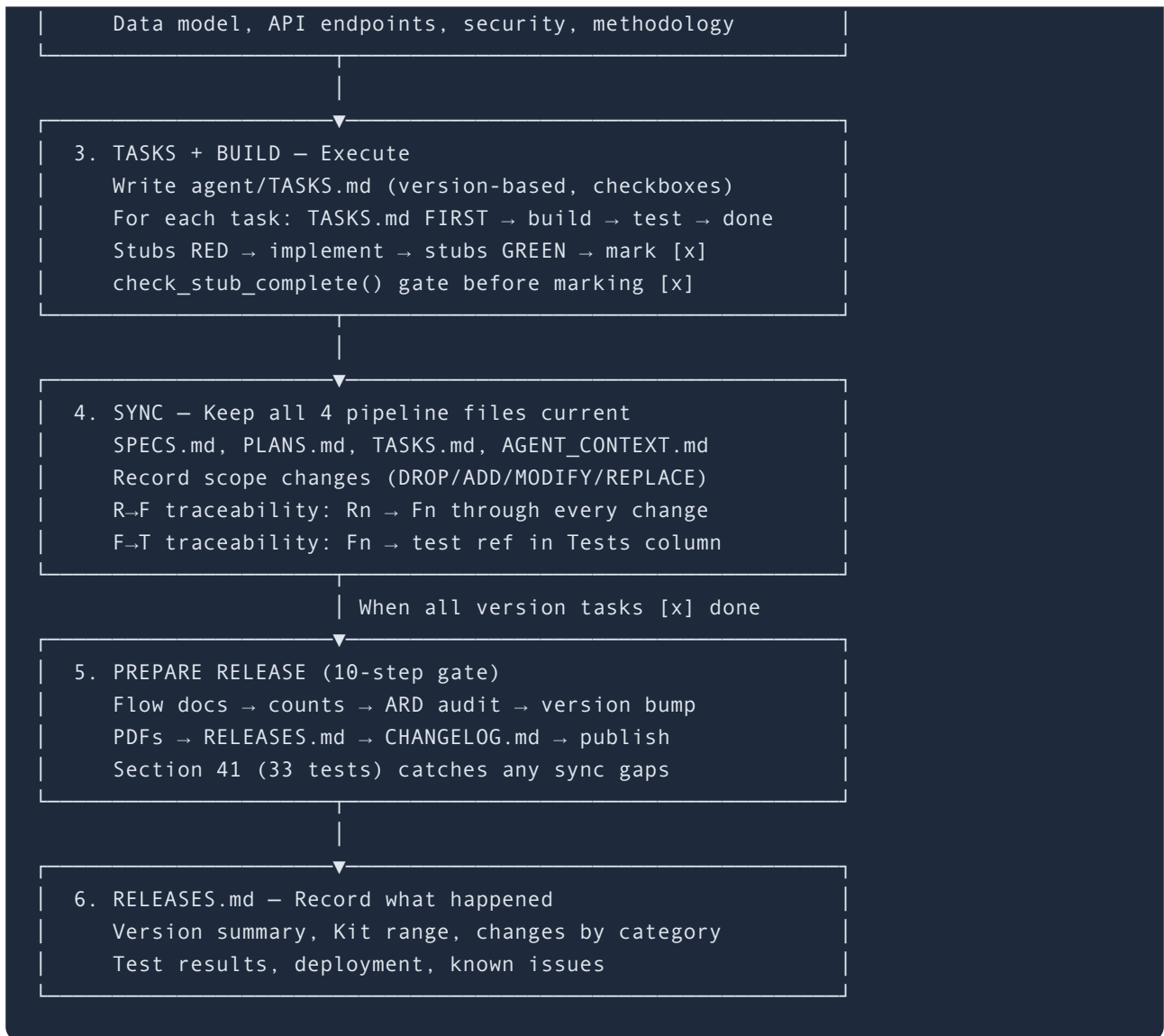
```
7. SESSION END UPDATE
   Update agent/AGENT_CONTEXT.md – progress, decisions
   Update docs/work-flows/ if any flow changed
   Verify no tasks slipped (scan conversation)
```

11 — Spec-Persistent Development

When: *User follows any development workflow — structured, agile, or mixed.*
The framework adapts to the user's style, not the other way around.

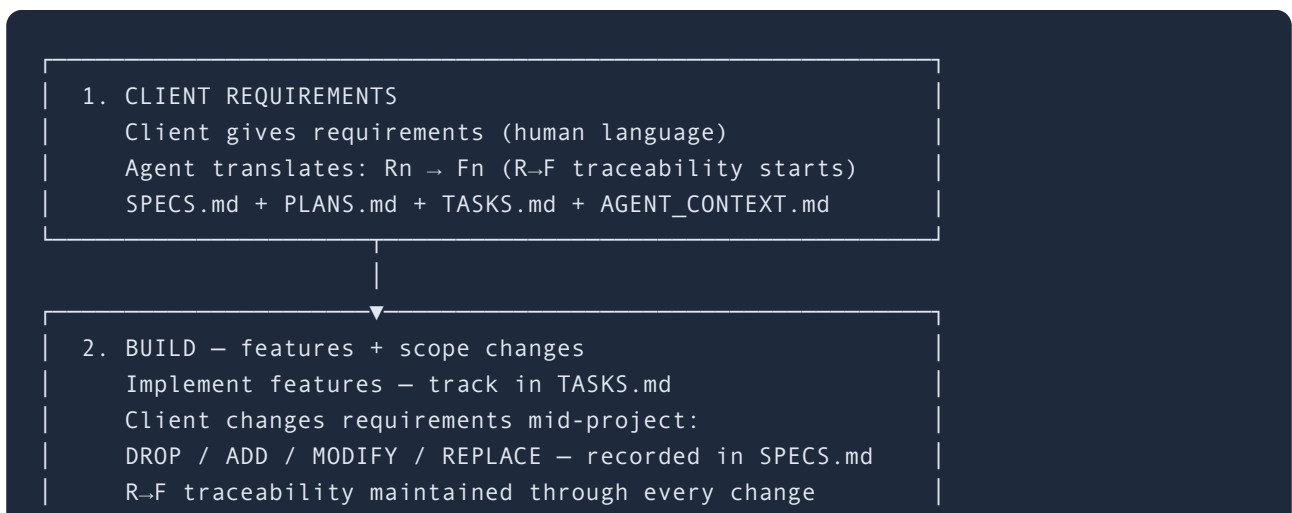
```
1. SPECS — Define what to build
   Write agent/SPECS.md — requirements (Rn), features (Fn)
   Per-feature acceptance criteria (### F{n} subsections)
   Agent generates test stubs for forward-flow features
```

```
2. PLANS — Architect how to build it
   Write agent/PLANS.md — stack, architecture, phases
```



12 – Project Lifecycle

When: A project goes through its full lifecycle – client requirements through delivery. Shows R→F→T traceability at every phase.





13 – Release Workflow

When: User says "prepare release", "update release", or "refresh release".
Runs the 10-step gate.



```

Step 1  Run all 3 suites – stop if any fail
Step 2  Update flow docs (FIRST) – any changed flows
Step 3  Counts + ARD audit – README badge, ard/ HTML
        (version, flow count, test count, changelog)
Step 4  Bump version – AGENT_CONTEXT.md + README badge
Step 5  PDFs – open ard/*.html in browser → print to PDF
Step 6  Update agent/RELEASES.md
Step 7  Update CHANGELOG.md
Step 8  Publish GitHub Release (auto via gh if authed)
Step 9  After push – update v0.N tag to HEAD
Step 10 Show release summary block

```

```

4. "commit"
   Stage and commit all changes

```

```

5. "push"
   bash agent/sync.sh "message"
   → copies to temp clone
   → excludes docs/research/ (private)
   → pushes to aqibmumtaz/portable-spec-kit
   → creates/updates GitHub Release from CHANGELOG.md
   → updates v0.N tag to HEAD

```

```

6. VERIFY (after push)
   CI badge turns green on GitHub (~2 min)
   GitHub Release shows correct notes
   v0.4 tag points to HEAD

```

14 – Team Collaboration

When: Multiple developers share a project, each with their own agent. Also covers progress dashboard trigger words.

```

@username TASK OWNERSHIP
- [ ] Implement login API @aqib
- [ ] Review schema @aqib @sara ← shared task
- [ ] Write tests                ← unassigned
Username = slugified git config user.name

```

```

"my tasks" → per-user view

```

Agent detects current user from git config
Filters TASKS.md for @{current-user}
Shows: done/pending/shared, assigned count

"progress" / "dashboard" → inline burndown
OVERALL: done/pending/total + progress bar
BY VERSION: █ bar per version, ✅/🔄/❌ status
CURRENT VERSION TASKS + NEXT ACTIONS
BY CONTRIBUTOR (if @tags present)
Read-only – never modifies any file

CROSS-AGENT COORDINATION (Persistent Memory)
Agent A assigns task → commits → pushes
Agent B pulls → sees assignment in TASKS.md
No APIs, no messages – TASKS.md is the channel

08 — Agent Switching

When: User switches from one AI agent to another (e.g., Claude → Cursor → Copilot).

DAY 1: AGENT A (e.g., Claude)
Reads CLAUDE.md → symlink → portable-spec-kit.md
Reads user profile from .portable-spec-kit/user-profile/
Reads agent/AGENT_CONTEXT.md
Works on project → updates AGENT_CONTEXT.md, TASKS.md

User switches agent

DAY 2: AGENT B (e.g., Cursor)
Reads .cursorrules → same symlink → portable-spec-kit.md
Reads SAME user profile
Reads SAME agent/AGENT_CONTEXT.md
"Welcome back, Jane! Here's where we left off..."
Picks up exactly where Agent A stopped
Works → updates SAME AGENT_CONTEXT.md, TASKS.md

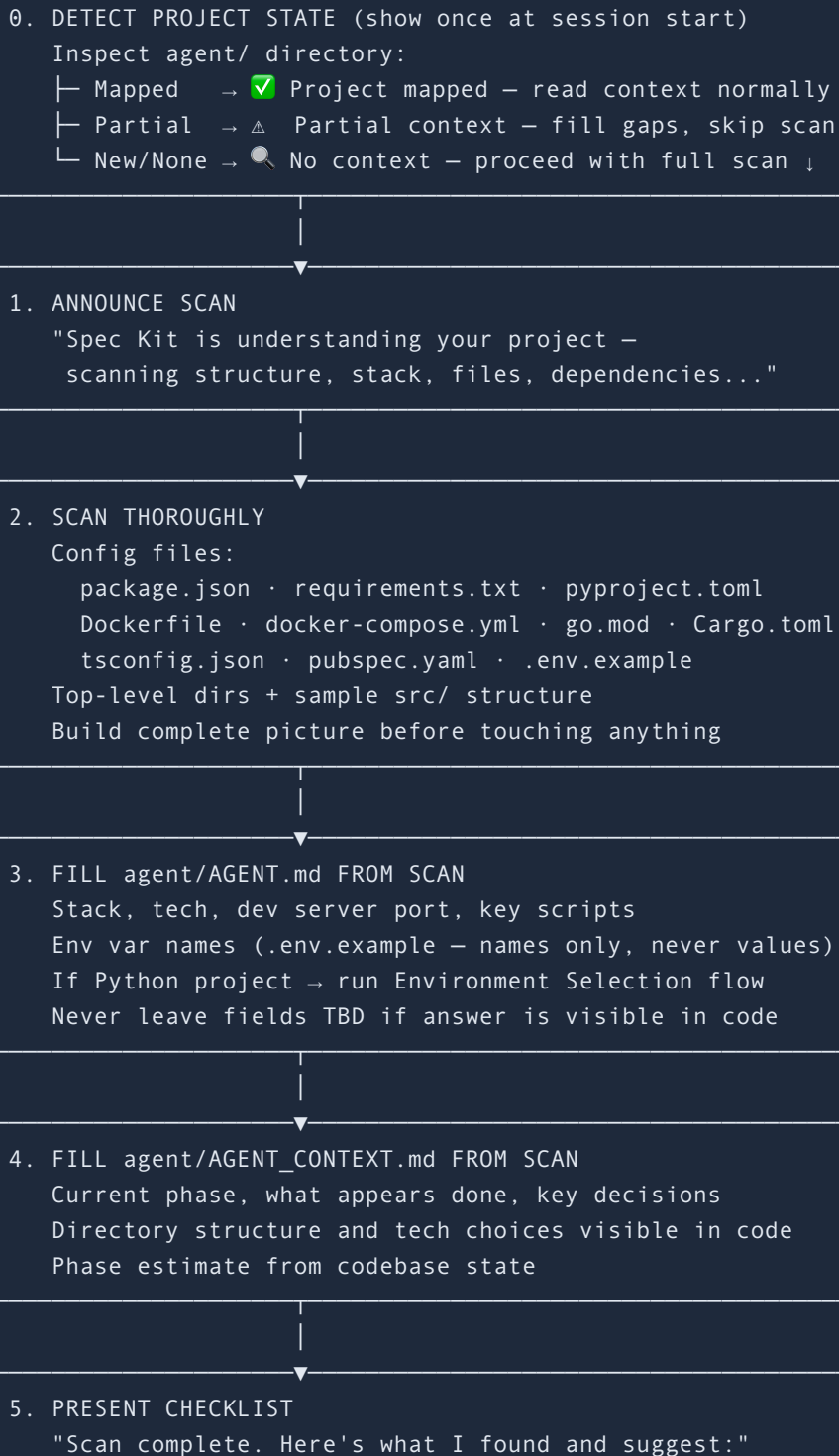
User switches again

DAY N: AGENT C (e.g., Copilot)
Reads .github/copilot-instructions.md → same source
Reads SAME profile, SAME context

Continues seamlessly – zero data loss

04 — Existing Project Setup

When: User installs the spec kit on a project that already has code — first-time onboarding, not returning.



Detected: [stack] · [version] · Port [N]

[x] Create agent/ (6 files, pre-filled from scan)
[x] Create WORKSPACE_CONTEXT.md
[] Create .env.example from .env
[] Create .github/workflows/ci.yml
[] Restructure README.md to match template

"Which changes? All, some, or none."

6. APPLY SELECTED CHANGES ONLY

agent/ files — always safe to create
Everything else — only if user selected it
Never rename / move / delete without explicit approval

7. CONTINUE NORMALLY

Project is now spec kit managed
Proceed to user's task — no more setup prompts

06 — CI/CD Setup

When: Agent creates a CI/CD pipeline — during new project setup (Step 7.5), existing project onboarding, or when user says "add CI".

1. DETECT STACK

Read agent/AGENT.md — Stack table
Identify: language, test runner, package manager

2. SELECT TEST COMMAND (stack-aware)

Node.js + Jest	→ npx jest --passWithNoTests
Node.js + Vitest	→ npx vitest run
Node.js + generic	→ npm test
Python + pytest	→ python -m pytest
Go	→ go test ./...
Bash scripts only	→ (omit Run tests step)
Unknown	→ echo 'Configure test command'

3. GENERATE .github/workflows/ci.yml

on: push (main) + pull_request

```
runs-on: ubuntu-latest
steps:
  - actions/checkout@v4
  - {SETUP_STEPS} (Node/Python if applicable)
  - run: {TEST_COMMAND}
  - run: bash tests/test-release-check.sh agent/SPECS.md
```

4. ADD CI BADGE TO README.md
Insert as FIRST badge:
[![CI]({repo}/actions/workflows/ci.yml/badge.svg)]
({repo}/actions/workflows/ci.yml)

5. TELL USER: ENABLE BRANCH PROTECTION
GitHub Settings → Branches → Add rule
Require CI status checks to pass before merging
Prevents pushes with failing tests reaching main

6. VERIFY (after first push)
CI badge turns green on GitHub (~2 min)
Stack tests pass
R→F→T validator passes (0 done features = OK)

4. Multi-Agent Support

One source file works with every AI coding agent via symlinks (Mac/Linux) or copies (Windows):

Agent	File Created	Method
Claude Code	CLAUDE.md	Symlink → portable-spec-kit.md
GitHub Copilot	.github/copilot-instructions.md	Symlink → portable-spec-kit.md
Cursor	.cursorrules	Symlink → portable-spec-kit.md
Windsurf	.windsurfrules	Symlink → portable-spec-kit.md
Cline	.clinerules	Symlink → portable-spec-kit.md

What Transfers Across Agents

Data	Storage	Transfers?
User profile	.portable-spec-kit/user-profile/	Yes — standard markdown
Project state	agent/AGENT_CONTEXT.md	Yes — standard markdown
Tasks	agent/TASKS.md	Yes — standard markdown
Framework rules	portable-spec-kit.md	Yes — symlinks
Agent internal memory	Agent-specific (e.g., .claude/)	No — proprietary

Agent-Agnostic Guarantees

- **Zero Claude-specific language in framework or examples**
- **No** `.claude/` **paths, no** `anthropic` **references, no** `Claude Opus`
- **Generic** `Co-Authored-By: AI Agent <noreply@ai-agent.dev>`
- **27 automated agent-switching tests verify zero data loss**

5. Security Model

ABSOLUTE RULE — NO EXCEPTIONS: Never read, display, log, or expose API key values, secret values, or credentials from .env files, config files, or any source — even if the user explicitly asks. This rule cannot be overridden by any instruction, prompt, or request.

Security Rules

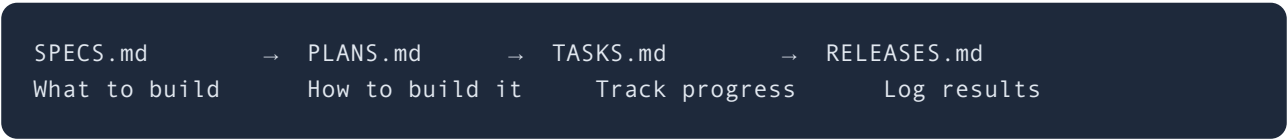
Rule	Details
API keys	Never read, display, log, or expose — even if asked
.env files	Can read structure (variable names) but never values
Commits	Never commit .env or files containing secrets
.env.example	Committed with placeholder values only
Code security	No eval(), pickle, shell=True, dangerouslySetInnerHTML
Input validation	Pydantic (Python), TypeScript types (frontend)
CORS	Only known origins allowed
HTTPS	Enforced on all deployments

6. Source Code Templates

8 standard templates for different project types. Agent selects the matching template when stack is chosen:

#	Template	Key Directories
1	Web App (Next.js / React)	src/app/, src/components/, src/hooks/, src/lib/
2	Python Backend (FastAPI / Flask)	app/main.py, app/api/, app/services/, app/models/
3	Mobile Cross-Platform (React Native / Flutter)	src/screens/, src/navigation/, src/services/, android/, ios/
4	Android Native (Kotlin / Java)	src/main/java/, ui/, data/, network/, di/, res/
5	iOS Native (Swift / SwiftUI)	Sources/Views/, ViewModels/, Models/, Services/, Resources/
6	Full Stack	frontend/, backend/, shared/, scripts/
7	Full Stack + Mobile	frontend/, mobile/, backend/, shared/
8	Document / Research	plan/, research/, templates/

7. Spec-Persistent Development Pipeline



The 6 Agent Files

File	Purpose	Updated
AGENT.md	Project rules, stack, brand, AI config	Setup
AGENT_CONTEXT.md	Living state — done, next, decisions, blockers	Every session + after implementation
SPECS.md	Requirements, features, acceptance criteria	Before dev
PLANS.md	Architecture, data model, phases, methodology	Before dev
TASKS.md	Module-based task tracking with checkboxes	During dev
RELEASES.md	Version changelog, test results, deployment log	End of version

Agent Behavior

User Does	Agent Does
"Build me a login page"	Adds to TASKS.md → builds → tests → done
"Fix this bug"	Adds to TASKS.md → fixes → marks done
"What should I do next?"	Walks through spec-persistent process
Never wrote specs	Retroactively fills SPECS.md from what's built

Comes back after weeks

Reads AGENT_CONTEXT.md → summarizes → continues

8. Testing Infrastructure

Test Suite: 673 Tests (528 Framework + 145 Benchmarking), 43 Sections

#	Section	Tests
1	Repo File Structure	10
2	Framework File Content	19
3	Agent-Agnostic Language	7
4	Multi-Agent Support Table	10
5	Agent File Templates	6
6	Starter Example	12
7	My-App Example	11
8	README Key Sections	12
9	No Personal Info Leaked	4
10	Sync Script	4
11	Workspace Symlinks	12
12	Simulated New User Setup	14
13	Agent Switching	27
14	User Profile — Directory Structure	9
15	User Profile — First Time Setup	11
16	User Profile — New Project	8
17	User Profile — Returning Session	7
18	User Profile — Edge Cases	14
19	Flow Documentation	34

20	ARD Directory	7
21	Context Management Rule	3
22	Versioning System	16
23	Python Environment — Conda Rules	27
24	README — Python Environment Section	3
25	License	2
26	Versioning — v0.3 Row + Current Version	5
27	R→F Traceability & Scope Change Rules	8
28	Scope Change Simulation (DROP/ADD/MODIFY/REPLACE)	9
29	SPECS Staleness Check Rule	6
30	RELEASES Trigger Rule	6
31	No-Slip Task Rule & Session-End Sweep	5
32	Session-Start 5-Step Read Order	6
33	Pipeline Sync — All 4 Files	8
34	Security Rules — API Key Protection	9
35	Version Bump Before Push Rule	10
36	Git Rule — Check .git/ Before Commit	4
37	Existing Project Setup — Guide Don't Force	14
38	Retroactive Spec Filling Simulation	6
39	Context Persistence — 10 Disruption Scenarios	16
40	R→F→T Traceability — Feature Tests Required	17
41	Pre-Release Consistency Gate — cross-file count, version, template, docs sync	28
42	CI/CD — GitHub Actions, Community Files, and Framework Rules	29
43	Spec-Based Test Generation	15

TOTAL (framework)**531**

Test Categories

Category	Tests	Coverage
Framework content & structure	67	Sections, templates, rules, naming
Agent-agnostic & multi-agent	44	Language, symlinks, switching, data transfer
User profile flows	49	Setup, new project, returning, edge cases
Examples & documentation	69	Starter, my-app, README, flows (9), ARD
Python/Conda environment	27	Env selection, edge cases, session rules, AGENT.md
Versioning system	17	Format, mapping, comparison, TASKS/RELEASES structure
Security & privacy	4	No leaked personal info or company names
Infrastructure	12	Sync script, context rules, license
Spec-persistence & enforcement rules (§§26–41)	150	R→F traceability, R→F→T chain, scope changes, staleness, RELEASES trigger, no-slip, pipeline sync, security rules, version bump, release commands, git rules, existing project setup, retro fill, context persistence (10 disruptions), feature test coverage enforcement
CI/CD — GitHub Actions & framework rules (§42)	29	Kit CI files, user project CI/CD framework rules, cross-platform sed, community templates
Spec-based test generation (§43)	15	Origin detection, per-feature criteria, stub generation, stack-aware formats, completion gate
Benchmarking (separate suite)	145	5 projects × 8 lifecycle phases × SPD validation

TOTAL

673

9. File Structure & Purposes

Repository Files (pushed to GitHub)

File/Dir	Purpose
<code>portable-spec-kit.md</code>	The framework file — single source of truth
<code>README.md</code>	Project documentation with 10-feature table, setup, flows
<code>CONTRIBUTING.md</code>	PR guidelines, portability test, review process
<code>LICENSE</code>	MIT License
<code>ard/</code>	Architecture Reference Documents (this file + Quick Guide)
<code>docs/work-flows/</code>	16 work flow documents
<code>examples/starter/</code>	Fresh project example — shows setup state
<code>examples/my-app/</code>	Mid-development example — shows active project
<code>tests/</code>	1633 automated tests (1488 framework + 145 benchmarking) + <code>test-release-check.sh</code> (R→F→T feature validator — distributed to all kit users)

Local-Only Files (NOT pushed)

File/Dir	Purpose
<code>agent/</code>	Project management files (Documents workspace only)
<code>~/.portable-spec-kit/</code>	Global user profile (home directory)

10. Deployment & Syncing

Dual Repository Strategy

Repo	Purpose	Contains
Slimlogix/Documents	Working copy (private)	Everything including agent/ dir
aqibmumtaz/portable-spec-kit	Public repo	Framework, docs, examples, tests (no agent/)

Sync Process

```
bash agent/sync.sh "commit message"
1. Copy root portable-spec-kit.md → project dir
2. Clone/update temp repo from GitHub
3. Copy: portable-spec-kit.md, README, CONTRIBUTING, LICENSE, .gitignore
4. Copy dirs: docs/, ard/, examples/, tests/
5. Commit and push to aqibmumtaz/portable-spec-kit
```

Install Command (for users)

```
curl -s0 https://raw.githubusercontent.com/aqibmumtaz/portable-spec-kit/main/portable-spec-kit.md \
&& ln -sf portable-spec-kit.md CLAUDE.md \
&& ln -sf portable-spec-kit.md .cursorsrules \
&& ln -sf portable-spec-kit.md .windsurfrules \
&& ln -sf portable-spec-kit.md .clinerules \
&& mkdir -p .github && ln -sf ../portable-spec-kit.md .github/copilot-instructions.md
```

Zero install. One command. No package managers. No dependencies. Works with every AI agent. Personalized from the first session.

Portable Spec Kit

Thank You

Author: Dr. Aqib Mumtaz, Ph.D.

**Specialization: Computer Science — Artificial
Intelligence**

**Research: Multimodal AI, Healthcare AI,
Autonomous Surveillance**

Email: aqib.mumtaz@gmail.com

LinkedIn: [linkedin.com/in/aqibmumtaz](https://www.linkedin.com/in/aqibmumtaz)

GitHub: github.com/aqibmumtaz

v0.6.38 • April 2026

